

Triads Future

SUMMARY

This document defines the development strategy, implementation order, project architecture, coding standards and deployment plan for Version 1 of the Triads Future website. The objective is to minimize development risk, avoid scope creep and ensure a stable production-ready release.

TECHNICAL SKILLS

Next.js (App Router)
TypeScript
Tailwind CSS
Framer Motion (minimal usage)
Next.js Route Handlers
Server Actions (where appropriate)
PostgreSQL
Supabase PostgreSQL
Local PostgreSQL
Prisma ORM
Session-based authentication
Role-based access
Cloudinary
Supabase Storage

PURPOSE

This document defines the development strategy, implementation order, project architecture, coding standards and deployment plan for Version 1 of the Triads Future website.

The objective is to minimize development risk, avoid scope creep and ensure a stable production-ready release.

DEVELOPMENT PHILOSOPHY

Version 1 is built with one objective:

Deliver a polished, maintainable and scalable product.

Development should prioritize: - Stability over complexity. - Reusability over duplication. - Performance over unnecessary features. - Completion over perfection.

Every feature added should have a clear business purpose.

SUGGESTED FOLDER STRUCTURE

app/ (website): page.tsx, solutions, work, insights, about, contact (cms): login, dashboard, appointments, work, insights, testimonials, faq, media, settings, profile api/ components/ features/ lib/ hooks/ services/ types/ utils/ styles/ public/ prisma/ docs/

FEATURE-BASED ARCHITECTURE

Organize code by feature rather than file type whenever practical.

Example features: - appointments - blog - portfolio - testimonials - faq

Each feature owns: - Components - Hooks - Validation - Types - Services

DEVELOPMENT ORDER

Phase 1: Project Setup (Create project, Configure TypeScript, Install Tailwind, Configure ESLint, Configure Prettier, Configure Prisma, Configure Database) Phase 2: Foundation (Global Layout, Fonts, Theme, Navigation, Footer, Responsive Grid, Shared Components) Phase 3: Homepage (Develop complete homepage, Test responsiveness, Optimize performance) Phase 4: Remaining Public Pages (Solutions, Work, About, Insights, Contact) Phase 5: CMS (Dashboard, Appointments, Work, Insights, Testimonials, FAQ, Media, Settings, Profile) Phase 6: Database Integration (Projects, Articles, Testimonials, Appointments, FAQ, Settings) Phase 7: SEO (Metadata, Open Graph, Structured Data, Sitemap, Robots, Canonical URLs) Phase 8: Animations (Only after layouts are stable) Phase 9: Testing (Desktop, Tablet, Mobile, CMS, Forms, SEO, Accessibility, Performance) Phase 10: Deployment (Production Build, Final Testing, Launch)

CODING STANDARDS

Every file should have one responsibility. Avoid large components. Separate UI, Logic, Database, Validation, Utilities.

COMPONENT RULES

Components should be reusable, typed, documented, accessible, responsive. Avoid duplication.

STATE MANAGEMENT

Prefer local state whenever possible. Use Context only for global requirements. Avoid unnecessary state libraries.

VALIDATION

Use schema validation. Validate forms, API requests, database input, uploads. Never trust client-side validation alone.

ERROR HANDLING

Every API should return success, meaningful error, and status code. Avoid generic error messages.

PERFORMANCE STRATEGY

Use Server Components where possible. Image optimization. Code splitting. Lazy loading. Dynamic imports. Caching where appropriate. Performance should be considered throughout development.

SEO CHECKLIST

Every page requires title, description, Open Graph, Twitter Card, canonical URL, structured headings, semantic HTML.

ACCESSIBILITY CHECKLIST

Keyboard navigation, alt text, focus states, ARIA labels where needed, semantic structure, readable contrast.

SECURITY CHECKLIST

Protected CMS routes, hashed passwords, environment variables, input validation, upload validation, authentication middleware.

TESTING CHECKLIST

Before deployment verify navigation, forms, CMS, responsive layouts, broken links, images, SEO, performance, accessibility, cross-browser support.

DEPLOYMENT CHECKLIST

Environment variables, database, storage, domain, HTTPS, metadata, favicon, sitemap, robots.txt, 404 page, 500 error page.

VERSION CONTROL

Recommended Git workflow: main → development → feature branches. Commit messages should be descriptive. Examples: feat: add appointment booking; fix: improve mobile navigation; refactor: simplify work card component.

SCOPE CONTROL

During development, never introduce new features. New ideas belong in ideas.md. The scope document is the source of truth.

DEFINITION OF DONE

A feature is complete only when functional, responsive, accessible, SEO ready, error handled, tested, and reviewed. Code complete does not mean feature complete.

FINAL DEVELOPMENT PRINCIPLE

Build the product that was planned. Not the product imagined halfway through development. Consistency, quality and completion are more valuable than adding one more feature before launch.